UPDATE

MCSL 384 Support Guide

PluginHive MCSL · Shopify / WooCommerce / BigCommerce / Magento

| Story ID | Story Title | Toggle Name | Trello card link |

Generated August 2026 · PluginHive QA Team

Included Story Cards

Story ID	Story Title	Toggle Name	Trello card link
ZI-629	WooCommerce Composite Products import [#350796]	21f0a963-e402-4c73-b8e5-ccb7ad11f5ac.woocommerce.bundle.classification.enabled, 21f0a963-e402-4c73-b8e5-ccb7ad11f5ac.woocommerce.composite.classification.enabled	ZI-629
ZI-630	WooCommerce role-restricted product import [#351838]	None	ZI-630
ZI-631	BlitzNow carrier integration [#392759]	None	ZI-631
ZI-632	AJEX carrier integration [#391852]	None	ZI-632
ZI-645	Canpar service renaming [#381046]	None	ZI-645
ZI-646	Delivro carrierCode missing — service fallback [#381046]	None	ZI-646
ZI-647	Delivro multiple labels generating for single order [#381046]	None	ZI-647
ZI-648	Delivro name translation (Deliveroo) [#381046]	None	ZI-648
ZI-650	WSS filter view cut off on scroll [#397045]	None	ZI-650
ZI-651	WSS order updates not syncing after edit [#375662]	{accountUUID}.wc.cancelled.order.reactivation.enabled	ZI-651
ZI-652	USPS REST Puerto Rico country code [#396604]	None	ZI-652
ZI-653	UPS third party billing in bulk actions [#396557]	None	ZI-653
ZI-654	Shipping price accumulation on edit [#395199]	None	ZI-654
ZI-655	Display customer-selected rate amount [#396457]	None	ZI-655
ZI-657	Bulk edit on products page [#394243]	{accountUUID}.products.bulk.edit.grid.enabled	ZI-657
ZI-658	Product page navigation [#394243]	None	ZI-658
ZI-661	PostNord currency conversion on zero-value [#394908]	None	ZI-661

Story ID	Story Title	Toggle Name	Trello card link
ZI-662	Vendor page not loading (Dokan) [#395822]	None	ZI-662
ZI-663	India Post label: print order ID in label [#385390]	None	ZI-663
ZI-664	Amazon-Referred Merchant Discount Plans (Other Platforms)	{accountUUID}.amazon.referred	ZI-664

ZI-629 - WooCommerce Composite Products import [#350796]

From SL: ZI-629 — WooCommerce Composite Products import [#350796]

Brief Description

MCSL 384 adds support for WooCommerce Composite Products (created via the WooCommerce Composite Products plugin) in MCSL. Previously, composite product types were not recognised during import, causing silent skips or incorrect packing behaviour. With this change, composite products import correctly into the MCSL Products list, and order line items reflect component-level "Shipped Individually" configuration — either as separate line items or as a single unit — for USPS and USPS Stamps label generation.

Toggles & Prerequisites

Toggle / config note	What support should confirm
`woocommerce.composite.classification.enabled`	Must be `true` for composite product/order classification to activate
`woocommerce.bundle.classification.enabled`	Must be `true`; confirm existing WooCommerce Product Bundles are unaffected (regression)
WooCommerce Composite Products plugin	Confirm merchant has this plugin installed and active on their WooCommerce store
Carrier prerequisite	USPS or USPS Stamps must be configured in hamburger menu > Carriers for label generation

Step-by-Step Support Walkthrough

Scenario 1 — Composite product imports and displays correctly

1. In WooCommerce Admin, go to MCSL > hamburger menu > Products and trigger a product import.
2. Confirm the composite product (e.g., "Create Your Own Gift Box") appears in the Products list with correct name and SKU (e.g., "Citrine & Navy 3×250ml").
3. Open the imported product record and verify name, SKU, weight, and dimensions match the WooCommerce product source.

Scenario 2 — Order import and label generation

4. In WooCommerce Admin, go to MCSL > ORDERS tab and open an order containing a composite product.
5. If components are set to **Shipped Individually**: confirm each component appears as a separate line item with its own weight and dimensions.
6. If components are **not** set to Shipped Individually: confirm the order shows the composite as a single line item using parent-level dimensions.
7. Open Prepare Shipment and confirm correct item count loads with no "invalid payload" error.
8. Click Generate Label via USPS Stamps and confirm the label generates successfully with no payload or packaging validation error.

Request/log fields to verify: _composite_parent, _composite_children meta keys present; classification logged as composite path, not plain/unknown product path.

Expected Behaviour

What support should observe	How to confirm
Composite product appears in MCSL Products list after import	hamburger menu > Products — name and SKU visible, no "unknown product type" warning
Line items reflect "Shipped Individually" setting per component	ORDERS tab > open order — separate line items or single unit depending on component config
Label generates without "invalid payload" error	Prepare Shipment and Generate Label complete successfully; tracking number visible in order view
WooCommerce Product Bundles (non-composite) continue to work	Regression: open a bundle order and confirm label generation still succeeds

ZI-630 - WooCommerce role-restricted product import [#351838]

WooCommerce role-restricted product import [#351838]

Brief Description

Prior to this fix, the MCSL WooCommerce plugin sent `status=publish` as a hardcoded query parameter when fetching products from the WooCommerce REST API. Products with role-based catalog visibility restrictions (e.g., wholesale-only) are published in WooCommerce but were silently excluded by this filter, causing them to be missing from the MCSL Products list on manual import. The fix removes that hardcoded parameter so role-restricted published products are imported correctly. Auto-import via webhook was not affected.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Fix is code-level; no setting to enable
WooCommerce only	This is not a Shopify issue — confirm merchant is on WooCommerce
Install third-party plugins	**Product Visibility by User Role for WooCommerce** — configures product visibility rules by user role. **User Role Editor** — creates custom user roles (e.g. Wholesale) needed to reproduce the affected product type
Manual import only	This issue applies only to manual product import. Before testing, disable auto product import via webhooks and test via manual import

Step-by-Step Support Walkthrough

Scenario 1 — Verify role-restricted products now import

1. In WooCommerce Admin, confirm at least one product exists with role-based catalog visibility restriction and published status.
2. Navigate to WooCommerce Admin → MCSL → Products and click **Import** to trigger a bulk product import.
3. After import completes, confirm the role-restricted product appears in the MCSL Products list alongside standard products.
4. Navigate to WooCommerce Admin → MCSL → Request Log and open the outbound product fetch request.

Request node to verify: confirm query string does not contain status=publish

5. Open an order containing the role-restricted product in WooCommerce Admin → MCSL → Orders; open the order and check Prepare Shipment — confirm weight and dimensions are populated, not blank or zero.

Scenario 2 — Regression: standard products unaffected

6. After the same bulk import, confirm all previously importable standard publicly visible products still appear in the MCSL Products list.
7. Confirm draft and private products are **not** present in the MCSL Products list.

Expected Behaviour

What support should observe	How to confirm
Role-restricted published products appear in MCSL Products list after manual import	Products list in WooCommerce Admin → MCSL → Products shows the product by name and SKU
`status=publish` absent from outbound API request	Request Log entry for the product fetch has no `status=publish` in the query string
Weight and dimensions populated on orders with role-restricted products	Prepare Shipment panel in MCSL Orders shows non-zero values sourced from the imported product
Standard publicly visible products import without regression	All products importable before the fix remain present after re-import

ZI-631 - BlitzNow carrier integration [#392759]

From SL: ZI-631 — BlitzNow carrier integration [#392759]

Brief Description

This card introduces BlitzNow as a new carrier integration within MCSL, covering registration, rate/proxy, label, and tracking flows. As of MCSL 384, the integration is blocked — no carrier module code exists yet, and test credentials are returning 401 errors. Support should treat this as an in-progress integration and not attempt to verify live functionality until the block is resolved and a build is confirmed deployed.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle detected	Confirm from live store/card context once integration ships
BlitzNow API credentials required	Credentials must be valid — current test credentials return 401; confirm with customer if escalating
New carrier module (no prior code)	Confirm a BlitzNow carrier option is visible under hamburger menu > Carriers before any testing
Release prerequisite	Confirm store is on MCSL 384 or the release that ships the completed build

Step-by-Step Support Walkthrough

Scenario 1 — Verifying carrier availability post-build

1. In Shopify Admin > Apps > PH Multi Carrier Shipping Label App, open hamburger menu > Carriers and confirm BlitzNow appears as a selectable carrier.
2. Attempt to connect a BlitzNow account using merchant API credentials; confirm no 401 or authentication error is returned.

Request/log fields to verify: authentication response status, credential acceptance confirmation

3. Place a test order in Shopify Admin > Orders and open it inside MCSL to trigger a rate request to BlitzNow.

Request/response nodes to verify: carrier request payload sent to BlitzNow proxy, rate response returned

4. Confirm rates display correctly in the MCSL order summary flow and a label can be generated.

Scenario 2 — Escalating the current blocked state

5. If BlitzNow does not appear under hamburger menu > Carriers, confirm the deployed MCSL build includes the completed carrier module — the card was blocked at 401 with zero carrier code at time of writing.
6. Collect the full request log from hamburger menu > Settings / Request Log and attach to the escalation ticket alongside the 401 response detail.

Expected Behaviour

What support should observe	How to confirm
BlitzNow visible as a carrier option	hamburger menu > Carriers shows BlitzNow listed
API credentials accepted without 401	Connection step completes without authentication error
Rates returned for test order	Rate response node present in request log
Tracking functional post-label	Tracking API returns status for generated BlitzNow shipment

ZI-632 - AJEX carrier integration [#391852]

From SL: ZI-632 — AJEX carrier integration [#391852]

Brief Description

MCSL 384 introduces AJEX as a new carrier integration on Shopify. Support agents can register an AJEX account, create domestic shipments (single and multi-package), generate labels, schedule pickups, and cancel shipments through the MCSL app. International shipments are not yet supported pending a product code from AJEX. Rates are also not supported for this carrier.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Feature is available in MCSL 384+ on Shopify
AJEX account required	Carrier must be configured under hamburger menu > Carriers with valid Client ID, Client Secret, and Customer Account
Environment toggle (productionKey)	Confirm whether store is on Sandbox or Production; sandbox shipments will not progress beyond INITIAL tracking status
International shipments	Not implemented — confirm customer is shipping domestically only

Step-by-Step Support Walkthrough

Scenario 1: Verify AJEX carrier setup and label generation

1. In Shopify Admin > Apps > PH Multi Carrier Shipping Label App, go to hamburger menu > Carriers and confirm AJEX is listed with Client ID, Client Secret, Customer Account, and Product Code populated.
2. Confirm Country of Origin is set to Saudi Arabia (SA) and the correct Pickup Method and Content Type are selected.
3. Go to the ORDERS tab, open a domestic order, and click Prepare Shipment — confirm AJEX appears as the carrier option.
4. Generate the label; confirm it is created successfully.

Request/response nodes to verify: receiver address fields (city, country) present in the label creation request; Short Address absent from request but visible on the printed label.

5. For multi-package orders, confirm all packages are listed before label generation and that a label is produced for each package.

Scenario 2: Verify pickup, manifest, and cancellation

6. Go to the PICKUP tab and confirm a pickup can be scheduled for an AJEX shipment.
7. From the ORDERS tab, confirm Manifest Generation completes without error for AJEX shipments.
8. Select a shipment and trigger cancellation; confirm the shipment status updates accordingly in the ORDERS tab.

Expected Behaviour

What support should observe	How to confirm
AJEX label generated for domestic orders	Label file downloads; order marked fulfilled in Shopify Admin > Orders
Tracking status shows INITIAL on sandbox	ORDERS tab > shipment detail; Retrack does not change status — expected on sandbox
Short Address absent from request, present on label	Request log shows no Short Address field; printed label displays it for sender and receiver
COD, insured, and large declared value shipments succeed	Generate labels for each type; confirm no error response from carrier

ZI-645 - Canpar service renaming [#381046]

From SL: ZI-645 — Canpar service renaming [#381046]

Brief Description

Canpar service display names were hardcoded as legacy strings in MCSL's static service maps. This card updates those display names to match Canpar's current official service names across all surfaces where a service name appears. The underlying service codes sent in API payloads are unchanged — this is a display-only change affecting Canpar (carrier C36) only.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Change is always-on after MCSL 384 is deployed
Canpar carrier configured	Confirm Canpar is active under hamburger menu > Carriers
Existing automation rules	Confirm rules using Canpar services still trigger correctly post-upgrade
Non-Canpar carriers	Confirm USPS, EasyPost, PostNord, Aramex display names are unaffected

Step-by-Step Support Walkthrough

Scenario 1 — Verify renamed services appear across UI surfaces (Shopify)

1. In Shopify Admin > Apps > PH Multi Carrier Shipping Label App, go to hamburger menu > Carriers and confirm Canpar is enabled.
2. Open a test order in the ORDERS tab, click into the order, and open Prepare Shipment — confirm the Canpar service dropdown shows updated official names, not legacy strings such as "CANPAR GROUND".
3. Proceed through Generate Label for one Canpar service and confirm the shipment confirmation screen displays the renamed service name.
4. From the ORDERS tab grid, use bulk label generation for a Canpar order and confirm the service name shown matches the new naming.
5. Go to hamburger menu > Settings > Automation (or Shipping Rates > Automation) and open an existing Canpar automation rule — confirm the service dropdown lists the renamed services and the rule still saves and triggers correctly.

Request/response nodes to verify: confirm the outbound API request still carries the original Canpar service code; confirm the response is mapped to the new display name in the ORDERS tab and on the generated label.

Scenario 2 — Verify historical orders and unmapped services

6. Open a historical Canpar shipment in the ORDERS tab and confirm the service name renders correctly without errors.
7. Confirm any unknown or unmapped Canpar service code does not throw an error — it should degrade gracefully.

Expected Behaviour

What support should observe	How to confirm
Updated Canpar service names appear in rate results, Prepare Shipment, and label confirmation	Check each surface in ORDERS tab on Shopify
Automation rules using Canpar services continue to fire without reconfiguration	Trigger a test order matching an existing Canpar rule and verify it applies
Outbound API service codes are unchanged	Check hamburger menu > Settings > Request Log — service code field must match pre-rename values
No impact on USPS, EasyPost, PostNord, or Aramex service names	Spot-check one rate or label for each carrier in ORDERS tab

ZI-646 - Delivro carrierCode missing — service fallback [#381046]

From SL: ZI-646 — Delivro carrierCode missing — service fallback [#381046]

Brief Description

When the Delivro carrier document in the database was missing the `carrierCode` field, `getRates.js` read it as `undefined`, causing `ratesApiHelper.js` to compose a broken `ruleId` and fail to return rates at checkout. A fallback was added in `carrierMapper.js` so that if `carrierCode` is absent or malformed, the carrier ID/name is used to infer the correct `ruleId`. This was a production config/code fix; no merchant-facing toggle was introduced.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Fix is live in MCSL 384 — confirm store is on this release
Delivro carrier document	Confirm the carrier document now has `carrierCode: "DELIVRO"` set, or that the fallback path is active
WooCommerce store	Validate on WooCommerce; fix is specific to this platform's reported incident
Other carriers co-configured	Confirm non-Delivro carriers are unaffected — no `ruleId` regression expected

Step-by-Step Support Walkthrough

Scenario 1 — Verify Delivro rates return at checkout

1. In the WooCommerce store admin, open the MCSL app and go to **hamburger menu > Carriers** — confirm the Delivro carrier entry exists.
2. Check whether `carrierCode` is present on the Delivro carrier document; note whether the store is relying on the direct value or the fallback path.
3. Place a test order for a Delivro-serviceable shipment and proceed to checkout — confirm Delivro rates appear with numeric prices and service labels.
4. Open **hamburger menu > Settings > Request Log** — locate the rate request for the test order.

Request/log fields to verify: ruleId — confirm it is fully formed (no undefined segment); also check for any warning-level entry indicating carrierCode was inferred via fallback.

Scenario 2 — Verify label generation and non-Delivro carrier regression

5. From **ORDERS**, select the test order and generate a label — confirm a single label is produced for a single package and multiple labels for multiple packages.
6. Cancel the shipment from the order summary flow and confirm cancellation succeeds.
7. Request rates for an order served by a non-Delivro carrier — confirm rates return normally and no `carrierMapper` fallback is logged for that carrier.
8. In the request log, confirm no `ruleId` anomaly appears for non-Delivro carriers.

Expected Behaviour

What support should observe	How to confirm
Delivro rates appear at checkout regardless of whether `carrierCode` is present in the carrier document	Live checkout test on WooCommerce store
`ruleId` in the rate request is fully formed — no `undefined` value in any segment	Request log in hamburger menu > Settings > Request Log
If fallback was used, a warning-level (not error-level) log entry references the carrier ID/name used for inference	Same request log entry — severity must be warning, not error
Non-Delivro carrier rate quotes and `ruleId` values are identical to pre-fix behaviour	Request log entries for other configured carriers show no `carrierMapper` fallback invocation

ZI-647 - Delivro multiple labels generating for single order [#381046]

From SL: ZI-647 — Delivro multiple labels generating for single order [#381046]

Brief Description

Prior to this fix, Delivro label generation in MCSL produced one label per product line item rather than one label per package. A WooCommerce order with 5 products × 3 quantity in a single package would generate 15 labels instead of 1. The fix corrects `itemsFor()` in `delivro/requestBuilder.js` to consolidate line items into package-level entries before sending the Delivro API request.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Fix is code-level; no merchant setting to enable
Delivro carrier required	Confirm Delivro is configured under hamburger menu > Carriers
WooCommerce platform	Reported and tested on WooCommerce; validate on the merchant's WooCommerce store
Packaging rules in use	Confirm packaging is configured under hamburger menu > Settings / Packaging if multi-package behaviour is being tested

Step-by-Step Support Walkthrough

Scenario 1 — Single package, multiple products (customer reproduction)

1. In WooCommerce Orders, open the affected order and go to ORDERS tab > open order > Prepare Shipment.
2. Confirm the order is packed into one package and Delivro is the selected carrier.
3. Click Generate Label and confirm exactly **1** label PDF is produced and **1** tracking number is stored on the order.

Request nodes to verify: confirm the outbound Delivro request in hamburger menu > Settings / Request Log shows a single consolidated package item — not one entry per line item or per unit quantity.

Scenario 2 — Multi-package order (1 label per package)

4. Open a WooCommerce order that splits into 2 packages under the configured packaging rules.
5. Select Delivro as the carrier and click Generate Label.
6. Confirm **2** label PDFs are produced, each with a distinct tracking number — label count must equal package count, not line-item count.

Request nodes to verify: Request Log should show one Delivro API call per package, each with its own consolidated item payload and a unique shipment/tracking ID in the response.

7. To verify no regression, repeat label generation on the same order type using a non-Delivro carrier and confirm label count matches that carrier's pre-fix behaviour.
8. Check that no duplicate or phantom shipment records appear on the WooCommerce order after label generation.

Expected Behaviour

What support should observe	How to confirm
1 label per package, not per line item or unit	Label PDF count on the order equals package count
Single tracking number per package stored on order	WooCommerce order detail shows no duplicate tracking entries
Consolidated package item in Delivro request payload	Request Log shows item array length = number of packages, not total units
Non-Delivro carrier label generation unchanged	Label count for other carriers matches pre-fix behaviour; `delivro/requestBuilder.js` not invoked

ZI-648 - Delivro name translation (Deliveroo) [#381046]

From SL: ZI-648 — Delivro name translation (Deliveroo) [#381046]

Brief Description

A customer reported seeing "Deliveroo" in their checkout, suspecting a PluginHive mistranslation of the carrier name "Delivro." Investigation confirmed no code change was needed: PluginHive consistently renders the carrier as "Delivro" across all definition sites. The "Deliveroo" string originates from a WooCommerce flat-rate shipping method the merchant configured themselves, not from PluginHive.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	No code change shipped; this is a support-resolved ticket
WooCommerce shipping zones	Merchant must have a flat-rate method manually named "Deliveroo" in their zone
Delivro carrier account	Confirm Delivro is connected under hamburger menu > Carriers
MCSL 384	Confirm store is on MCSL 384 or later if verifying release context

Step-by-Step Support Walkthrough

Scenario 1 — Confirm PluginHive renders "Delivro", not "Deliveroo"

1. In WooCommerce Admin, open the PluginHive MCSL app and navigate to hamburger menu > Carriers — confirm the carrier is listed as **Delivro**.
2. Navigate to ORDERS tab > open an affected order > Prepare Shipment — confirm the carrier label shown reads **Delivro**.
3. If a label has been generated, download the label PDF and confirm **"Deliveroo" does not appear** anywhere on the PluginHive-generated output.

Request/response nodes to verify: carrier name field should read "Delivro" / DELIVRO in the Delivro API request and response

4. Check hamburger menu > Settings > Request Log for the relevant order — confirm the outbound carrier name is **Delivro**, not Deliveroo.

Scenario 2 — Isolate the merchant's WooCommerce flat-rate method as the source

5. In WordPress Admin, go to WooCommerce > Settings > Shipping > Shipping Zones — locate the zone and inspect the flat-rate method names configured by the merchant.
6. Confirm a method named **"Deliveroo"** exists there and was created by the merchant, not by PluginHive.
7. Ask the merchant to rename or remove that flat-rate method, then have a test customer refresh checkout — confirm the "Deliveroo" label disappears while PluginHive-sourced **"Delivro"** rates remain.

Expected Behaviour

What support should observe	How to confirm
PluginHive settings and carrier list show "Delivro" only	hamburger menu > Carriers — no "Deliveroo" string present
Checkout "Deliveroo" entry traces to merchant's flat-rate method	WooCommerce Shipping Zones shows merchant-created method named "Deliveroo"
API request/response carries carrier name "Delivro" / DELIVRO	Request Log in hamburger menu > Settings confirms outbound node value
Removing merchant's flat-rate method removes "Deliveroo" from checkout	Test checkout after merchant deletes/renames the WooCommerce method

ZI-650 - WSS filter view cut off on scroll [#397045]

From SL: ZI-650 — WSS filter view cut off on scroll [#397045]

Brief Description

The WSS filter panel on the MCSL orders page was being clipped or cut off when the user scrolled the orders list in WooCommerce admin. This fix ensures the filter panel remains fully visible in the viewport throughout scrolling. The change is UI/CSS-level and carrier-neutral. Note: QA flagged a known fail state — if the store has a product, import, or onboarding banner present, the panel visibility fix may not hold.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Fix is always active in MCSL 384 on WooCommerce
Onboarding/import/product banner present	QA marked this scenario as FAIL — confirm whether the merchant's store shows any such banner before troubleshooting
Browser zoom level	QA verified at 100%, 125%, 150% — confirm zoom level if merchant reports clipping
Cross-browser	QA verified across supported browsers — confirm browser if merchant reports clipping

Step-by-Step Support Walkthrough

Scenario 1 — Verify filter panel visibility during scroll (no banners)

1. Log into WooCommerce admin and open the MCSL app; navigate to the **ORDERS** tab.
2. Confirm no onboarding, import, or product banner is displayed at the top of the page.
3. Click the WSS filter icon to open the filter panel.
4. Scroll the orders list to the bottom and back to the top — confirm the filter panel remains fully visible with no clipping at any edge.
5. Apply a filter, scroll 500px, then click **Apply** — confirm the orders list refreshes to matching results; click **Clear** and confirm the list resets.

Scenario 2 — Verify behaviour when an onboarding/product/import banner is present

6. With a banner visible on the MCSL ORDERS page, open the WSS filter panel and scroll — confirm whether clipping reoccurs.
7. If clipping is observed with a banner present, capture a screenshot and escalate; this is the known QA FAIL condition for this card.

Expected Behaviour

What support should observe	How to confirm
Filter panel stays fully visible during scroll (no banner)	Scroll orders list top-to-bottom; no edge of the panel is clipped
Apply and Clear controls remain functional after scrolling	Apply a filter post-scroll; orders list filters and resets correctly

What support should observe	How to confirm
Filter panel clips when onboarding/import/product banner is present	Reproduce with banner active — this is the documented QA FAIL state
No clipping regression on non-WSS filter panels	Open a non-WSS admin filter panel and scroll; behaviour unchanged

ZI-651 - WSS order updates not syncing after edit [#375662]

From SL: ZI-651 — WSS order updates not syncing after edit [#375662]

Brief Description

When a WooCommerce order was cancelled and then moved back to Processing, the MCSL WSS sync was not re-importing the order — leaving it stuck in a previous batch and ineligible for label generation. This fix introduces a toggle-gated reactivation path: when a CANCELLED sub-order is detected during sync, the order falls through to a full re-import instead of patching in place. Affects WSS order sync on WooCommerce only.

Toggles & Prerequisites

Toggle / config note	What support should confirm
`{accountUUID}.wc.cancelled.order.reactivation.enabled`	Must be ON for reactivation fall-through to trigger; confirm it is set for the merchant's account UUID
Kill-switch takes precedence over the enabled toggle	If both enabled and disabled flags are set, legacy diff behaviour applies — confirm only the enabled flag is active
Toggle OFF	Sync falls back to legacy order-diff behaviour; no reactivation occurs and no fall-through log line is emitted
WooCommerce only	This path is not applicable to other platforms

Step-by-Step Support Walkthrough

Scenario 1 — Cancelled order reactivated and re-imported

1. Confirm the reactivation toggle is ON for the merchant's account UUID before testing.
2. In WooCommerce Orders, cancel an order that has already been imported to WSS, then move it back to **Processing**.
3. Wait approximately 60–75 seconds for the WSS sync to fire.
4. In MCSL **ORDERS** tab, locate the order and confirm it appears active with the current WooCommerce shipping address and line items.

> **Request/log fields to verify: sync log line reading `"has 1 CANCELLED sub-order(s) — falling through for reactivation"` confirms the new code path ran.**

5. Attempt **Prepare Shipment / Generate Label** on the reactivated order and confirm no batch-association error blocks label generation.

Scenario 2 — Toggle OFF or kill-switch active (legacy path)

6. With the toggle OFF (or kill-switch set), repeat steps 2–3.
7. In MCSL **ORDERS** tab, confirm the order shows an order-diff record rather than a re-import; no fall-through log line should appear.
8. Confirm a never-cancelled Processing order still syncs address changes in place within ~60 seconds (regression check via hamburger menu > **Request Log**).

Expected Behaviour

What support should observe	How to confirm
Reactivated order appears in WSS ORDERS grid with current Woo address and line items	Check MCSL ORDERS tab after ~60–75 s; no manual re-import needed
Sync log contains fall-through message for CANCELLED sub-order	hamburger menu > Request Log — look for `"has 1 CANCELLED sub-order(s) — falling through for reactivation"`
Label generation succeeds on reactivated order with no batch-association error	Prepare Shipment / Generate Label completes without error in MCSL ORDERS tab
Never-cancelled orders continue to sync in place; no fall-through log line emitted	Confirm via Request Log — patch fires once, no reactivation message present

ZI-652 - USPS REST Puerto Rico country code [#396604]

From SL: ZI-652 — USPS REST Puerto Rico country code [#396604]

Brief Description

USPS REST was classifying US ↔ Puerto Rico shipments as international because the `isInternational()` logic used strict country-code inequality (`originCountry !== destinationCountry`), and Puerto Rico carries the country code `PR`. This fix corrects that classification so US ↔ PR shipments are treated as USPS domestic for both rate requests and label generation. Affects USPS REST on Shopify and WooCommerce.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Fix is code-level; no merchant setting to enable
USPS REST carrier configured	Confirm USPS REST (not USPS legacy) is active under hamburger menu > Carriers
Shopify or WooCommerce store on MCSL 384+	Confirm plugin version is 384 or later before troubleshooting
Puerto Rico destination uses country code PR	Confirm the order/address has PR as country, not US

Step-by-Step Support Walkthrough

Scenario 1 — Rates returning for US ↔ Puerto Rico

1. In the MCSL app, go to hamburger menu > Carriers and confirm USPS REST is the active USPS carrier.
2. On WooCommerce or Shopify, locate a test or live order with destination country `PR` (e.g. ZIP 00901).
3. Open the order in the ORDERS tab > Prepare Shipment and trigger a rate fetch.
4. Confirm USPS domestic services appear (e.g. Priority Mail, Ground Advantage) and no international-only services are listed.

Request/response nodes to verify: confirm the rate request does not include international service codes; response should return domestic service names only.

Scenario 2 — Label generation for US ↔ Puerto Rico

5. With a domestic USPS rate selected for a US-to-PR order, click Generate Label in the ORDERS tab.
6. Confirm the label generates successfully and no customs form is required or attached.

Request/log fields to verify: check the request log (hamburger menu > Settings > Request Log) to confirm the shipment is not flagged as international in the outbound USPS REST payload.

7. Regression check: open a US-to-Canada order and confirm USPS international services are still returned and customs fields remain required.

Expected Behaviour

What support should observe	How to confirm
USPS domestic rates returned for US ↔ PR	Rate list in Prepare Shipment shows domestic services; no international services present
Label generates without customs form for US ↔ PR	Label document in ORDERS tab reflects domestic service; no customs attachment
US ↔ Canada still returns international rates	Regression order to Canada shows international services and customs fields intact
Request log does not classify PR shipment as international	Outbound USPS REST payload in hamburger menu > Settings > Request Log shows domestic routing

ZI-653 - UPS third party billing in bulk actions [#396557]

From SL: ZI-653 — UPS third party billing in bulk actions [#396557]

Brief Description

MCSL 384 adds a new **"Set UPS Third Party Billing"** bulk action to the ORDERS grid, allowing merchants to apply a third-party billing account (account number, postal code, and country) across multiple UPS orders in one operation. Previously, third-party billing was only configurable via automation rules or per-order settings. Only orders in `INITIAL` or `PROCESSING` status are eligible; `SHIPPED` and `LABEL_CREATED` orders are silently excluded from the batch.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Feature is available in MCSL 384 and above on Shopify
UPS carrier must be configured	Confirm UPS is active under hamburger menu > Carriers
Bulk action only appears after order selection	Merchant must select at least one order before the Bulk Actions menu exposes this option
Eligible statuses: `INITIAL` / `PROCESSING` only	Orders in `SHIPPED` or `LABEL_CREATED` are silently excluded — confirm from order status column

Step-by-Step Support Walkthrough

Scenario 1 — Applying third-party billing via bulk action

1. In Shopify Admin > Apps > PH Multi Carrier Shipping Label App, go to the **ORDERS** tab.
2. Select one or more UPS orders in `INITIAL` or `PROCESSING` status using the order grid checkboxes.
3. Open **Bulk Actions** and confirm **"Set UPS Third Party Billing"** appears in the menu.
4. Click the action; confirm the dialog opens with fields: **Account Number**, **Billing Postal Code**, **Billing Country**.
5. Enter valid values (e.g., account `V45V34`, postal `98055`, country `United States` or `Canada`) and click **Submit**.

Request node to verify: POST /bulkaction/upsthirdpartybilling body → { data: { accountNumber, billingPostalCode, billingCountry }, orders: [...] }; UPS payload → ShipmentCharge.BillThirdParty

6. Confirm the success alert reads **"Updated N of N orders"** and the grid re-renders.

Scenario 2 — Ineligible or missing selection

7. Select only `SHIPPED` orders and attempt the action; confirm the alert reads **"UPS Third Party Billing can only be set for orders with status - Initial or Processing"** and the dialog does not open.
8. With zero orders selected, confirm the alert reads **"Please select one or more orders to proceed"** and no dialog opens.

Expected Behaviour

What support should observe	How to confirm
Success alert shows "Updated N of N orders"; grid refreshes	Visible in ORDERS tab immediately after submit
UPS label shows `BILLING: 3RD PARTY` (domestic) or `BILLING: P/P TPS <account> <country>` (international)	Print label from the affected order after bulk action is applied
`ShipmentCharge.BillThirdParty` contains the submitted account number, postal code, and country	Verify in hamburger menu > Settings > Request Log for the UPS label request
Empty Account Number, Postal Code, or Country blocks submit with "This field is required"; no network call fires	Attempt submit with a blank field in the dialog

ZI-654 - Shipping price accumulation on edit [#395199]

From SL: ZI-654 — Shipping price accumulation on edit [#395199]

Brief Description

When a merchant edits shipping on a Shopify order (delete + re-add), MCSL was reading `total_shipping_price_set` — which accumulates all historical shipping lines — instead of summing only the active `shipping_lines[]` entries. This caused over-declared freight on FedEx commercial invoices, creating customs compliance risk. MCSL 384 fixes the Shopify order mapper to filter out removed lines (`is_removed !== true`) before calculating the shipping total.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Fix is applied automatically in MCSL 384 for all qualifying Shopify orders
Shopify Auto Order Updates	Confirm auto-sync is enabled; edited shipping propagates to MCSL within ~60 s without manual reprocessing
FedEx international label with commercial invoice	Fix is relevant when a commercial invoice is generated; WooCommerce and BigCommerce invoice paths are unaffected
Shopify Edit Order used by merchant	Confirm the merchant edited shipping via Shopify Edit Order, not a third-party tool

Step-by-Step Support Walkthrough

Scenario 1 — Verify corrected freight on a Shopify FedEx order with edited shipping

1. In Shopify Admin, open the affected order and confirm the shipping line history shows a deleted line and one active line.
2. In MCSL ORDERS tab, open the same order and check the **Shipping Fee Paid** value in the order grid — it must equal the active shipping line only, not the sum of all historical lines.
3. Open **Prepare Shipment / Generate Label** for the order and proceed to generate a FedEx international label with commercial invoice.
4. On the generated commercial invoice PDF, locate the **Freight** cell and confirm it matches the active Shopify shipping line converted at the displayed exchange rate.

Request/response nodes to verify: shipping_lines[].price (active only), shipping_lines[].is_removed, total_shipping_price_set — confirm MCSL used the filtered active line value, not total_shipping_price_set.

5. If the merchant reports the old accumulated value still appears, confirm MCSL 384 is the installed release and that auto-sync completed (allow up to 60 s after the Shopify edit).

Scenario 2 — Verify unedited single-line orders are unaffected (regression check)

6. In MCSL ORDERS tab, open an order with no shipping edit history (single active line).
7. Confirm the MCSL order details shipping value, commercial invoice Freight cell, and packing slip freight all match the single active shipping line — behaviour must be identical to pre-fix output.

Expected Behaviour

What support should observe	How to confirm
MCSL order grid **Shipping Fee Paid** equals the active shipping line only	Compare MCSL grid value against the active `shipping_lines[].price` in Shopify Admin
Commercial invoice **Freight** cell reflects active line (currency-converted), not accumulated total	Open generated CI PDF; value must not equal sum of removed + active lines
`total_shipping_price_set` is not used as the shipping source for edited orders	Check request log — freight node must match active-line sum, not `total_shipping_price_set`
WooCommerce and BigCommerce commercial invoice freight values are unchanged	Confirm on the reported platform that invoice freight equals the platform's current canonical shipping value

ZI-655 - Display customer-selected rate amount [#396457]

From SL: ZI-655 — Display customer-selected rate amount [#396457]

Brief Description

MCSL 384 adds a read-only **Customer selected rate** field to the order details page in the MCSL app on Shopify. It surfaces the shipping rate the customer chose at checkout (carrier name + amount) directly inside the order, so support and merchants no longer need to cross-reference Shopify Admin to identify what the customer paid for shipping. The field is display-only and has no effect on rate shopping or label generation. Carriers covered: FedEx, UPS, DHL, USPS.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Feature is always active in MCSL 384+ on Shopify
Shopify store on MCSL 384	Confirm app version before troubleshooting missing field
Checkout rate must have been captured at order time	Orders placed before this release, draft orders, or orders with no `selectedShippingRate` will show blank or N/A — this is expected
Display-only field	Confirm merchant understands this value does not pre-select or influence rate shopping

Step-by-Step Support Walkthrough

Scenario 1 — Verifying the field displays correctly

1. In Shopify Admin, open the MCSL app and go to the **ORDERS** tab.
2. Open an order placed after the MCSL 384 release where the customer selected a carrier rate at checkout.
3. On the order details page, locate the **Customer selected rate** field and confirm it shows carrier name and amount (e.g., `UPS Ground — $12.45`) in the store's checkout currency.
4. Open a second order for a different carrier (FedEx, DHL, or USPS) and confirm the field reflects that carrier's rate and amount.

Scenario 2 — Verifying the field is display-only and does not affect label generation

5. On an order showing a populated **Customer selected rate**, click **Prepare Shipment / Generate Label**.
6. Confirm the rate list is freshly computed — no rate is silently pre-selected to match the displayed value.
7. Select a different rate, generate the label, and confirm the label reflects the manually chosen rate, not the displayed customer-selected value.
8. Open a pre-release order or draft order in the **ORDERS** tab and confirm **Customer selected rate** renders as blank or N/A with no errors or missing sections elsewhere on the page.

Expected Behaviour

What support should observe	How to confirm
Customer selected rate field visible on order details page	Field shows carrier name and amount for post-384 orders with a captured checkout rate
Blank or N/A for orders with no captured rate	Pre-release orders, draft orders, and orders missing `selectedShippingRate` show blank/N/A — page still loads fully
Field is read-only	No input control present; value cannot be edited or saved back
Label generation unaffected	Rate shopping in Prepare Shipment shows freshly computed rates; displayed customer rate is not pre-selected

ZI-657 - Bulk edit on products page [#394243]

From SL: ZI-657 — Bulk edit on products page [#394243]

Brief Description

MCSL 384 adds a bulk-edit capability to the Products page in the MCSL app, controlled by a per-account feature toggle. When enabled, merchants can select multiple products and edit their dimensions across all selected products in a single action. This is carrier-neutral and applies to the Shopify implementation of MCSL.

Toggles & Prerequisites

Toggle / config note	What support should confirm
`{accountUUID}.products.bulk.edit.grid.enabled: true`	Must be enabled for the account UUID; if absent or false, bulk-edit UI will not appear
MCSL release prerequisite	Store must be on MCSL 384 or later
No carrier prerequisite	Feature is carrier-neutral

Step-by-Step Support Walkthrough

Scenario 1 — Bulk edit is enabled and working

1. In the MCSL app, go to **hamburger menu > Products**.
2. Confirm row checkboxes are visible; select two or more products.
3. Click the **Edit Products** button that appears in the toolbar after selection.
4. Verify the bulk-edit dialog opens and lists all selected products with their variants.
5. Edit dimensions for the selected products and click to apply.

Request nodes to verify: dimension fields in the product update payload for each selected product ID.

6. Confirm the Products grid reflects the updated dimensions after save, and that unedited products are unchanged.

Scenario 2 — Toggle is off or bulk-edit UI is missing

7. If the merchant reports the **Edit Products** button is not visible, confirm the toggle `{accountUUID}.products.bulk.edit.grid.enabled` is set to `true` for their account UUID.
8. If the toggle is confirmed on and the button is still absent, confirm the store is on MCSL 384 and escalate with the account UUID and a screenshot of the Products page.

Expected Behaviour

What support should observe	How to confirm
Checkboxes appear on each product row when toggle is enabled	Visible in hamburger menu > Products
Edit Products button appears after selecting one or more products	Toolbar state changes on selection

What support should observe	How to confirm
All selected products and their variants appear in the bulk-edit dialog	Visible in the dialog before applying
Saved dimensions are reflected in the Products grid immediately after apply	Reload hamburger menu > Products and inspect the updated rows

ZI-658 - Product page navigation [#394243]

From SL: ZI-658 — Product page navigation [#394243]

Brief Description

Previously, navigating from the Products page into a product detail view and returning (via browser back or in-app breadcrumb) would reset all applied filters and search terms, because the component cleared state on every mount. This card fixes that behaviour so filter chips and search terms are preserved on return. The fix is carrier-neutral and applies to the hamburger menu > Products area in MCSL on Shopify.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Fix is always active in MCSL 384+
Shopify store on MCSL 384	Confirm release version before troubleshooting
No carrier prerequisite	Applies to all products regardless of carrier assignment

Step-by-Step Support Walkthrough

Scenario 1 — Browser back preserves filter and search state

1. In the MCSL app, go to hamburger menu > Products.
2. Apply at least one filter (e.g. Product type) and enter a search term in the search box.
3. Click into any product row to open its detail view.
4. Press the browser back button to return to the Products page.
5. Confirm the filter chip, search term, and grid rows are identical to what was set before navigating in.

Scenario 2 — Clearing filters after return works cleanly

6. With preserved filters visible after returning (per steps 1–5), click **Clear filters** and clear the search box.
7. Confirm all filter chips are removed, the search box is empty, and the grid reloads to the default unfiltered product list with no stale rows.
8. Open a fresh browser session, navigate to hamburger menu > Products, and confirm no filters or search terms are pre-applied.

Expected Behaviour

What support should observe	How to confirm
Filter chips survive navigation back from product detail	Chips visible and active on Products page after browser back or in-app breadcrumb return
Search term survives navigation back	Search box still contains the pre-navigation term and grid reflects it
Clear filters removes all preserved state	After clearing, grid shows default unfiltered view with no residual chips or rows

What support should observe	How to confirm
Fresh session starts clean	New browser session shows Products page with no filters or search pre-applied

ZI-661 - PostNord currency conversion on zero-value [#394908]

From SL: ZI-661 — PostNord currency conversion on zero-value [#394908]

Brief Description

On BigCommerce stores, PostNord shipment requests for zero-value orders were sending `rate: 1` (hardcoded) alongside the store's native currency (e.g., AUD or USD) instead of SEK. This caused `runRateConversionToSEK` to fail or produce incorrect declared values. The fix ensures that when `order.total` is 0, both `rate` and `currency` are forced to `1` and `SEK` respectively in the PostNord shipment payload. Non-zero orders and WooCommerce PostNord flows are unaffected.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Fix is always active for BigCommerce PostNord label generation in MCSL 384+
BigCommerce only	WooCommerce PostNord path is explicitly not changed — confirm platform before investigating
PostNord carrier required	Confirm PostNord is the active carrier on the order being tested
Zero-value order condition	Confirm `order.total = 0`; non-zero orders follow the existing conversion path

Step-by-Step Support Walkthrough

Scenario 1 — Zero-value BigCommerce PostNord order

1. In BigCommerce admin, locate a zero-value order and open it in the MCSL ORDERS tab.
2. Click **Prepare Shipment / Generate Label** and select PostNord as the carrier.
3. Open hamburger menu > **Request Log** immediately after label generation.
4. Inspect the outbound PostNord shipment request payload.

Request nodes to verify: rate: 1 and currency: "SEK" present at shipment level; no store currency (e.g., AUD, USD) on that field

5. Confirm the generated label shows a declared value of **SEK 1.00**.

Scenario 2 — Non-zero BigCommerce order (regression check)

6. Open a BigCommerce PostNord order with a positive order total in the MCSL ORDERS tab.
7. Generate the label and open hamburger menu > **Request Log**.
8. Confirm the payload uses the store currency and that `rate` reflects the converted SEK amount via the existing conversion path — not hardcoded to `1`.

Request nodes to verify: currency matches store currency; rate is derived from order total, not hardcoded

Expected Behaviour

What support should observe	How to confirm
Zero-value BigCommerce PostNord payload always emits `rate: 1`, `currency: "SEK"`	Request log — shipment-level nodes
Label for zero-value order shows declared value SEK 1.00	Inspect generated label document
Non-zero orders use existing currency conversion, not the hardcoded values	Request log — `rate` reflects converted amount, not `1`
WooCommerce PostNord label generation is unchanged	Generate a WooCommerce PostNord label; no SEK-forcing behaviour in request log

ZI-662 - Vendor page not loading (Dokan) [#395822]

From SL: ZI-662 — Vendor page not loading (Dokan) [#395822]

Brief Description

On WooCommerce stores using the Dokan multi-vendor plugin, the MCSL vendor page was failing to load for Dokan vendors due to two compounding bugs: a JWT audience mismatch on outbound API calls and incorrect handling of the `isSpMvEnabled` subscription flag. This fix corrects the JWT `aud` claim and the MV login flow so vendor-scoped pages render correctly. Non-Dokan stores and standard WooCommerce admin views are unaffected.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Fix is code-level; no setting to enable
Dokan plugin must be installed and active	Confirm Dokan is present on the merchant's WooCommerce site
Multi-vendor (MV) subscription must be active (`isSpMvEnabled = true`)	Confirm merchant has an active MV subscription in MCSL
Non-Dokan stores should follow standard MCSL load path	Confirm no MV-specific endpoints are called on non-Dokan sites

Step-by-Step Support Walkthrough

Scenario 1 — Dokan vendor page loads correctly

1. Log into the WooCommerce site as a Dokan vendor and open the MCSL vendor page from the Dokan dashboard.
2. Confirm the page renders with the vendor's own orders list visible; no blank screen or indefinite spinner.
3. Open browser DevTools > Network tab and capture outbound XHR/fetch requests to MCSL vendor APIs.

Request nodes to verify: Authorization: Bearer JWT — decode and confirm the aud claim matches the value expected by the MCSL backend; all requests return HTTP 200

4. Confirm only that vendor's own orders and shipments are listed (cross-check against WooCommerce Orders in WordPress Admin).

Scenario 2 — MV disabled or non-Dokan regression

5. Log in as the WooCommerce site admin (non-vendor) and open the MCSL app; confirm the standard admin view loads with all vendors' orders visible and no vendor-scoped filtering applied.
6. On a non-Dokan store, open MCSL and check the browser console/network tab to confirm no MV-specific endpoints are called and no JWT audience errors appear.
7. If a vendor reports a blank/hanging page, confirm `isSpMvEnabled` status in MCSL — if false, the page should display a clear disabled/subscription-required message, not hang.

Expected Behaviour

What support should observe	How to confirm
Dokan vendor page renders with vendor-scoped orders	Page loads within ~5 seconds; orders list visible in Dokan dashboard
JWT `aud` claim is correct on all vendor API calls	Network tab — decode Bearer token; all vendor API requests return HTTP 200
MV-disabled vendors see a clear message, not a blank page	Log in as vendor with `isSpMvEnabled = false`; confirm message displayed, no spinner
Non-Dokan / admin views unaffected	Standard MCSL admin view loads normally; no MV JS initialisation in console

ZI-663 - India Post label: print order ID in label [#385390]

From SL: ZI-663 — India Post label: print order ID in label [#385390]

Brief Description

India Post booking and label API requests now include the `bkg_ref_id` field, populated with the Shopify order name (e.g., `#1234`). Previously this field was never sent, so the order reference did not appear on the printed label. The change affects India Post labels only — other carriers are unaffected. Both single and bulk label generation are covered.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No toggle	Feature is always active for India Post in MCSL 384+
India Post carrier configured	Confirm India Post is set up under hamburger menu > Carriers
Shopify order name must be present	If order name is blank, label generation should still succeed — confirm graceful handling
ZPL thermal printers	QA note: QR code does not print on ZPL; `bkg_ref_id` reference field is unaffected — confirm with merchant printer type if raised

Step-by-Step Support Walkthrough

Scenario 1 — Single label

1. In Shopify Admin > Apps > PH Multi Carrier Shipping Label App, go to the ORDERS tab and open the target order.
2. Click **Prepare Shipment / Generate Label** and select India Post as the carrier.
3. Generate the label and download the PDF — confirm the reference area on the label shows the Shopify order name (e.g., `#1234`).
4. In hamburger menu > Settings > Request Log, locate the outbound India Post booking request for this order.

Request nodes to verify: article.bkg_ref_id (must equal the Shopify order name), article.bulk_reference (must still be present and unchanged)

Scenario 2 — Bulk label

5. From the ORDERS tab, select multiple India Post orders and trigger bulk label generation.
6. Download the bulk label PDF and confirm each label shows its own unique order name in the reference area.
7. In the request log, open the bulk booking request and verify each article entry carries its own `bkg_ref_id` value matching the corresponding order name.

Request nodes to verify: article[n].bkg_ref_id for each article — no duplicates, no missing values

8. Generate a FedEx label for any order and confirm the FedEx request log contains no `bkg_ref_id` field.

Expected Behaviour

What support should observe	How to confirm
India Post label PDF shows Shopify order name in the reference area	Open downloaded label PDF — single and bulk
`bkg_ref_id` present in India Post booking and label API request, value matches Shopify order name	Check request log node `article.bkg_ref_id`
`bulk_reference` field still present alongside `bkg_ref_id` with its original value	Check same request log article object — both fields must coexist
FedEx (and other carrier) payloads unchanged — no `bkg_ref_id` field present	Inspect FedEx request log entry for the same order

ZI-664 - Amazon-Referred Merchant Discount Plans (Other Platforms)

From SL: ZI-664 — Amazon-Referred Merchant Discount Plans (Other Platforms)

Brief Description

This card introduces Amazon-branded discounted subscription plans for merchants referred to MCSL via Amazon Shipping, controlled by a per-account toggle. When the toggle is ON, the merchant sees Amazon-branded plans at discounted pricing instead of the standard plan catalogue. The feature is confirmed for WooCommerce, BigCommerce, and Magento; Shopify merchants do **not** see Amazon-referred plans even if the toggle is ON.

Toggles & Prerequisites

Toggle / config note	What support should confirm
`{accountUUID}.amazon.referred` — per-account toggle	Must be ON in the merchant's account config for Amazon plans to appear; confirm with engineering or account owner
Platform scope	Feature applies to WooCommerce, BigCommerce, and Magento only — Shopify is explicitly excluded
Carrier prerequisite	Amazon Shipping must be the relevant carrier context for these plans
Payment gateway	Stripe, PayPal, and Razorpay are confirmed to apply discounted pricing at checkout

Step-by-Step Support Walkthrough

Scenario 1 — Toggle ON: Amazon-referred merchant sees discounted plans

1. Confirm `{accountUUID}.amazon.referred` is ON for the merchant's account (check with engineering or account config).
2. On the reported platform (WooCommerce, BigCommerce, or Magento), open the MCSL app and navigate to the Subscription / Plans view.
3. Verify Amazon-branded plan rows or badges are visible and prices are numerically lower than standard plans.
4. Have the merchant proceed to checkout for a plan; confirm the discounted amount is shown on the checkout confirmation screen.
5. After purchase, confirm the subscription details and billing record reflect the discounted amount, not the standard price.

Scenario 2 — Toggle OFF or Shopify: standard plans only

6. If the toggle is OFF, or the merchant is on Shopify, open the Subscription / Plans view on the reported platform.
7. Confirm no Amazon-branded tier or discount badge appears and only the standard plan catalogue is displayed.
8. If a merchant reports losing Amazon pricing after a toggle change mid-subscription, flag to engineering — post-purchase toggle-flip behaviour is an open product question per card AC.

Expected Behaviour

What support should observe	How to confirm
Amazon-branded discounted plans visible only when toggle is ON and platform is WooCommerce, BigCommerce, or Magento	Plans view in MCSL app on the reported platform
Discounted price shown at checkout and stored in billing/invoice record	Checkout confirmation screen and subscription details view
Shipment and carrier limits of the Amazon plan enforced post-subscription	Confirm from live store or escalate to engineering with account UUID
Existing non-Amazon subscriptions unaffected by this change	Check active subscription details for unrelated merchants on the same release